

Desenvolvimento Back-End



Aula 03

- Métodos;
- Sobrecarga de métodos;
- Métodos Construtores;

Métodos

Classe =

Atributos (Propriedades)



Métodos (Comportamento)



Propriedades:

- ✓ marca
- ✓ modelo
- ✓ Sistema operacional

Comportamento:

- ✓ liga
- ✓ desliga
- ✓ envia mensagem

Celular
marca: string modelo: string so: string
ligar(): void desligar(): void enviarMensagem(msg: string): void

Métodos

São conjuntos de instruções que cumprem uma tarefa específica e devolvem ou não um valor ao programa que o chamou.

```
tipo de retorno nome (parâmetros)
{
    instruções;
    <return resposta>;
}
```

Métodos

- Os métodos também possuem padrão de nomenclatura. Eles sempre começam com a primeira inicial minúscula e as outras iniciais maiúsculas.
 - **pegarInformacao()**
 - **executarComandoInicial()**
- Os métodos geralmente são ações que podem ser efetuadas entre os atributos do objeto.
- Métodos que não retornam valores, apenas executam ações, são do tipo *void*.

Métodos

- Os métodos podem ou não assumir tipos de dados, caso não assumam, são chamados de **procedimentos**, pois executam um conjunto de instruções sem devolverem valor algum a quem os chamou. Um método sem tipo recebe em sua definição a palavra-chave **void** no lugar do tipo.

```
//Procedimento sem parâmetro  
void imprime() {  
    System.out.println("Estamos imprimindo!");  
}
```

Métodos

- Quando os métodos assumem algum tipo, eles são chamados de **funções** e precisam do comando **return** para devolver o valor resultante da execução de suas instruções internas.

```
//Função sem parâmetro  
int numero() {  
    int num = Math.random();  
    return num;  
}
```

Métodos

- Os métodos podem receber dados para serem utilizados internamente, os quais são chamados de **parâmetros** ou de **argumentos**.
- Quando os parâmetros são passados para os métodos, é criada uma cópia dos valores.

```
//Procedimento com parâmetro  
void dobro (int n) {  
    int resp;  
    resp = n * 2;  
    System.out.println(resp);  
}
```

```
//Função com parâmetro  
int soma(int n, int m) {  
    int s;  
    s = n + m;  
    return s;  
}
```

Métodos

- Podemos passar vários parâmetros para os métodos, inclusive de tipos diferentes.

```
//Procedimento com parâmetro  
void linha (int n, char ch) {  
    System.out.println(ch);  
    System.out.println(n);  
}
```


Métodos

```
//definição das funções  
float quad (float z) {  
    return (z * z);  
}  
float soma (float m, float n) {  
    return (m + n);  
}  
float somaquad (float m, float n) {  
    return soma(quad(m), quad(n));  
}
```

Uso do método *quad* como
parâmetro do *soma*

Sobrecarga de Métodos

- Podemos criar vários métodos com o mesmo nome em uma classe, as ações são parecidas, mas os **parâmetros** necessários devem ser diferentes;
- **Sobrecarregar** um método é criar mais de um método com o **mesmo nome** mas **com parâmetros diferentes**;
- Os parâmetros podem se diferenciar em tipo e/ou em quantidade, de modo a alterar a **assinatura** do método.

Sobrecarga de Métodos

Assinaturas Diferentes

Assinatura do método é composta por:

- 1) Nome
- 2) Lista de parâmetros

```
public void print( int i ) { ... }  
public void print( float f ) { ... }  
public void print( String s ) { ... }  
public void print( String s, float g ) { ... }  
public void print (String s, int i) {...}
```

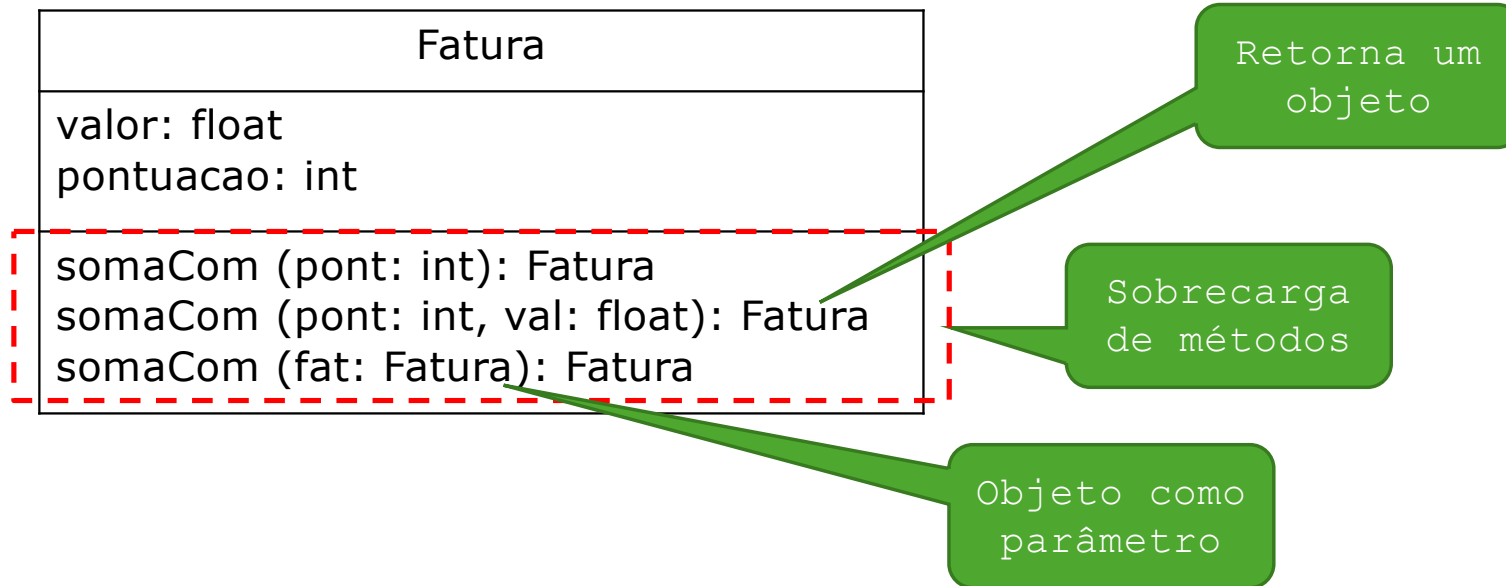
```
public int print (int i) {...}
```

É sobrecarga?

Exemplo:

```
public class Sobre carga {  
    public int soma (int a) {  
        return a;  
    }  
    public int soma (int a, int b) {  
        return a+b;  
    }  
    public int soma (int a, int b, int c) {  
        return a+b+c;  
    }  
}
```

Exemplo



Exemplo 01

Conta
numero: int agencia: int saldo: double
Conta(n: int, a: int) Conta(n: int) imprimeDados(): String sacar(vlr_saque: double):boolean depositar(vlr_deposito: double):boolean

O valor só poderá ser retirado da conta caso exista saldo disponível, caso contrário mostre a mensagem “Saque não permitido”

Exemplo 02

Boletim
n1: double n2: double media: double
Boletim(n1: double, n2: double) imprimeBoletim(): void calculaMedia(): void verificaConceito(): string

Calcula a média:
 $media = (n1 + n2) / 2$

Retorna o conceito, conforme o valor da media, de acordo com a tabela ao lado

MÉDIA	CONCEITO
8,0 —●—●— 10,0	A
6,0 —●—○— 8,0	B
4,0 —●—○— 6,0	C
Média abaixo de 4	D

Construtores

- São trechos de código invocados automaticamente quando um objeto é instanciado.
- NUNCA retornam nada e não possuem tipo.
- Os construtores sempre tem o mesmo nome da classe a que se refere.
- Para cada objeto, o construtor é chamado exatamente uma vez, na sua criação.

Exemplo:

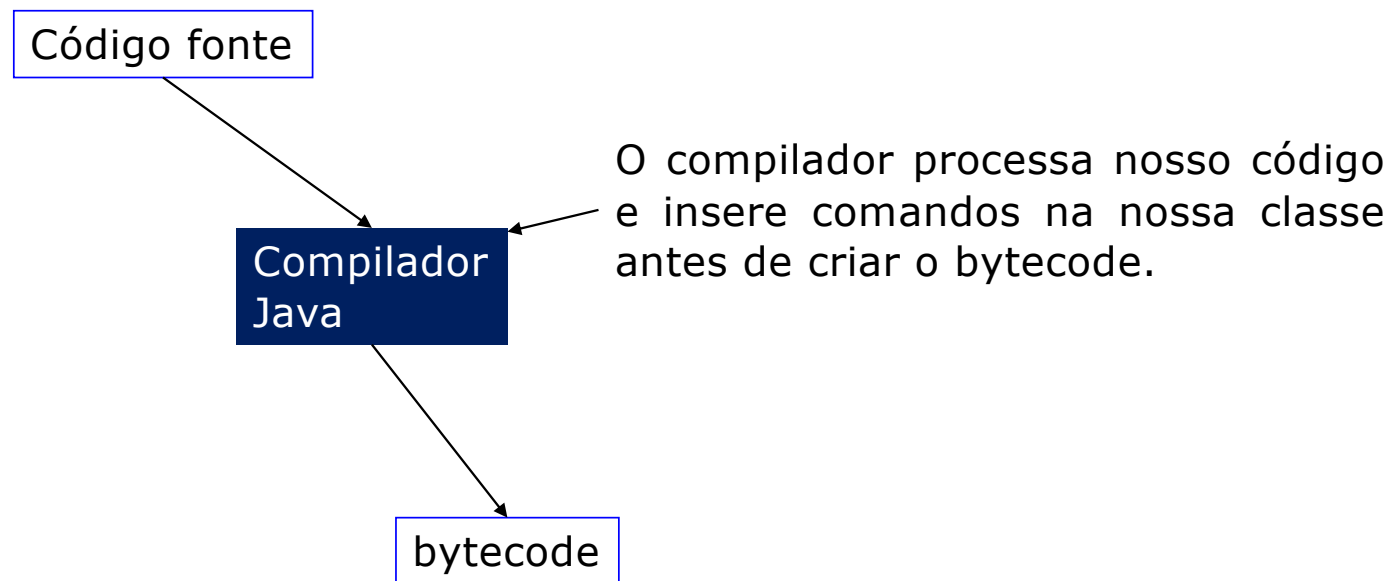
– Objeto obj = new Objeto();

Alguns podem requerer parâmetros

– Objeto obj1 = new Objeto(35, "Nome");

Construtores (observação)

- Se não colocarmos nenhum construtor na nossa classe, o compilador do Java irá inserir o construtor padrão em nosso código antes de gerar o bytecode, assim como outras alterações que veremos no decorrer o semestre.



Construtores (observação)

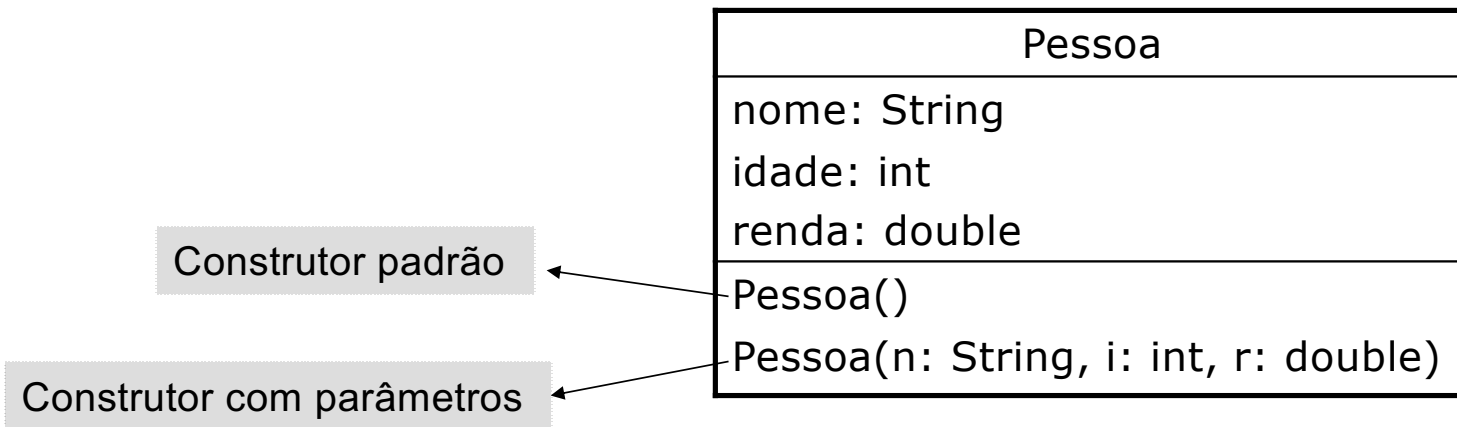
Se não tem construtor, o Java insere um.

```
public class Pessoa {  
    String nome;  
    int idade;  
    double renda;  
  
}
```



```
public class Pessoa extends Object{  
    String nome;  
    int idade;  
    double renda;  
  
    Pessoa(){  
  
    }  
}
```

Exemplo



Construtores - Exemplos

```
public class Pessoa {  
    String nome;  
    int idade;  
    double renda;  
  
    Pessoa() {  
        System.out.println("O construtor foi acionado");  
    }  
}
```

O construtor obrigatoriamente possui o mesmo nome da classe

```
public class UsaPessoa {  
    public static void main(String[] args) {  
        Pessoa p = new Pessoa();  
    }  
}
```

O construtor é acionado somente quando a classe é instanciada

```
-----Configur  
O construtor foi acionado
```

A mensagem é exibida porque o construtor **Pessoa** foi acionado na instânciação da classe

Construtores - O que é errado

```
public class UsaPessoa {  
    public static void main(String[] args) {  
        Pessoa p = new Pessoa();  
        p.Pessoa();  
    }  
}
```

Gera erro chamar um construtor como se ele fosse um método

```
public class Pessoa {  
    String nome;  
    int idade;  
    double renda;  
    void Pessoa() {  
        System.out.println("O construtor foi acionado");  
    }  
}
```

Colocar um tipo de retorno no construtor, implica em alterar a assinatura dele e com isso, ele não será acionado quando a classe for instanciada

Construtores com parâmetros/argumentos

- Alterando o construtor para inicializar os atributos da classe

```
public class Pessoa {  
    String nome;  
    int idade;  
    double renda;  
  
    Pessoa(String n, int i, double r) {  
        nome=n;  
        idade=i;  
        renda=r;  
    }  
}
```

Um construtor pode ser alterado para receber tantos parâmetros quanto o programador quiser, no entanto, na instânciação da classe têm que ser passadas estas informações

```
public class UsaPessoa {  
    public static void main(String[] args) {  
        Pessoa p = new Pessoa("Fulano", 40,1000.0);  
        System.out.println("Nome: "+p.nome);  
        System.out.println("Idade: "+p.idade);  
        System.out.println("Renda: "+p.renda);  
    }  
}
```

```
-----  
Nome: Fulano  
Idade: 40  
Renda: 1000.0  
  
Process completed.
```

Construtores com parâmetros/argumentos

- O que é errado ao ter um construtor que recebe parâmetros

```
public class UsaPessoa {  
    public static void main(String[] args) {  
        Pessoa p = new Pessoa();  
    }  
}
```

Gera erro: criar uma instância da classe sem passar os argumentos requeridos

```
public class UsaPessoa {  
    public static void main(String[] args) {  
        Pessoa p = new Pessoa("Fulano");  
    }  
}
```

Gera erro: criar uma instância da classe sem passar todos os argumentos requeridos

```
public class UsaPessoa {  
    public static void main(String[] args) {  
        Pessoa p = new Pessoa("Fulano", 12.0, 1000.0);  
    }  
}
```

Gera erro: criar uma instância da classe e passar argumentos de tipos incompatíveis, no exemplo, float no lugar de int

Classes com mais de um construtor (Sobrecarga)

```
public class Pessoa {  
    String nome;  
    int idade;  
    double renda;  
  
    Pessoa(){  
        System.out.println("Construtor acionado");  
    }  
  
    Pessoa(String n, int i, double r) {  
        nome=n;  
        idade=i;  
        renda=r;  
    }  
}
```

Uma classe pode ter mais de um construtor, desde que, exista diferença na quantidade e tipos de parâmetros recebidos.

Neste exemplo, um construtor não recebe nenhum parâmetro e o outro recebe três, logo, o sistema pode diferenciar um do outro no momento da instanciação

Todos os construtores obrigatoriamente tem de ter o mesmo nome da classe e não ter tipo de retorno

Classes com mais de um construtor

```
public class UsaPessoa {  
    public static void main(String[] args) {  
  
        Pessoa p = new Pessoa("Fulano", 40, 1000.0);  
  
        System.out.println("Nome: "+p.nome);  
        System.out.println("Idade: "+p.idade);  
        System.out.println("Renda: "+p.renda);  
  
        Pessoa p1 = new Pessoa();  
  
        System.out.println("\nNome: "+p1.nome);  
        System.out.println("Idade: "+p1.idade);  
        System.out.println("Renda: "+p1.renda);  
    }  
}
```

A classe pode ser instanciada de duas maneiras, uma não passando nenhum parâmetro e outra passando três (nesta ordem: String, int e double)

Em cada uma das maneiras de instanciar é chamado o construtor correspondente

```
-----  
Nome: Fulano  
Idade: 40  
Renda: 1000.0  
Construtor acionado  
  
Nome: null  
Idade: 0  
Renda: 0.0  
  
Process completed.
```

Exemplos

Triangulo
base: float altura: float
Triangulo() Triangulo(b: float, a: float)

```
public class Triangulo {  
    float base;  
    float altura;  
  
    Triangulo(){}  
  
    Triangulo(float b, float a){  
        base = b;  
        altura = a;  
    }  
}
```

Data
dia: int mes: int ano: int
Data() Data(d: int, m: int, a: int)

```
public class Data {  
    int dia;  
    int mes;  
    int ano;  
  
    Data(){ }  
    Data(int d, int m, int a){  
        dia = d;  
        mes = m;  
        ano = a;  
    }  
}
```

Usando a classe

```
public class TesteClasses {  
  
    public static void main(String[] args) {  
        Triangulo t = new Triangulo(2.5f, 3f);  
        float area = (t.base * t.altura)/2;  
        System.out.println("Área: " + area);  
  
        Data d1 = new Data(2, 9, 2015);  
        Data d2 = new Data();  
        System.out.println(d1.dia+"/"+d1.mes+"/"+d1.ano);  
        System.out.println(d2.dia+"/"+d2.mes+"/"+d2.ano);  
    }  
}
```

Exercícios

1 - Considere os diagramas UML dos seguintes itens, observe que cada item possui 2 construtores distintos. Construa o código das respectivas classes e crie 2 objetos usando os construtores criados e os preencha com dados fornecidos pelo usuário. Exiba os dados dos objetos. Obs. Utilize encapsulamento.

Paciente
nome: string rg: string endereco: string telefone: string dataNascimento: string profissao: string
Paciente() Paciente(nome: string)

Exercícios

2 - Considere os diagramas UML dos seguintes itens, observe que cada item possui 2 construtores distintos. Construa o código das respectivas classes e crie 2 objetos usando os construtores criados e os preencha com dados fornecidos pelo usuário. Exiba os dados dos objetos. Utilize encapsulamento.

Produto
marca: string fabricante: string cod_barras: string preco: float
Produto() Produto (m: string, f:string, c:string, p:float)

Já sei tudo?

- Só isso? Já sei tudo que diz respeito à orientação a objetos?
- Não, ainda há um longo caminho pela frente, com muitos conceitos importantes ;
 - Abstração, Herança, Polimorfismo etc.
- Agora que tudo começa a ficar realmente divertido! 😬 😊 (é sério!)





That's all Folks!